```
import pandas as pd
import numpy as np
df = pd.read_csv('kaggle_dataset.csv')
```

1. Exploratory Data Analysis (EDA)

```
df = pd.read_csv('kaggle_dataset.csv')
df.head(10)
```

	feature_1	feature_2	feature_3	feature_4	feature_5	feature_6	feature_7	feature_8	feature_9	feature_10	...	f
0	15000.00	62	1	0	2240.50	70.0000	104.2857	45	52	1.2155	...	
1	50024.53	61	1	0	496.72	1522.0000	439.1429	1	1	0.0266	...	
2	32770.00	34	0	1	1489.81	186.4286	0.0000	11	11	0.7938	...	
3	82003.32	33	1	0	NaN	NaN	NaN	0	0	NaN	...	
4	11057.77	68	1	0	261.20	7.8571	32.2857	17	22	0.5889	...	
5	11210.74	70	1	0	69.45	176.8571	523.1429	2	2	0.0464	...	
6	59115.32	61	1	0	217.67	291.0000	78.4286	4	4	0.0890	...	
7	23615.07	68	1	0	50.33	21.5714	1060.4286	3	4	0.1153	...	
8	10838.32	43	0	1	393.91	246.4286	0.0000	4	4	0.2399	...	
9	24732.82	45	1	0	216.15	7.8571	1135.0000	2	2	0.0857	...	

10 rows × 87 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38985 entries, 0 to 38984
Data columns (total 87 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   feature_1   38985 non-null  float64
 1   feature_2   38985 non-null  int64
 2   feature_3   38985 non-null  int64
 3   feature_4   38985 non-null  int64
 4   feature_5   36771 non-null  float64
 5   feature_6   36771 non-null  float64
 6   feature_7   36771 non-null  float64
 7   feature_8   38985 non-null  int64
 8   feature_9   38985 non-null  int64
 9   feature_10  36771 non-null  float64
 10  feature_11  38985 non-null  float64
 11  feature_12  38985 non-null  float64
 12  feature_13  36771 non-null  float64
 13  feature_14  36771 non-null  float64
 14  feature_15  23258 non-null  float64
 15  feature_16  26095 non-null  float64
 16  feature_17  28240 non-null  float64
 17  feature_18  23258 non-null  float64
 18  feature_19  26095 non-null  float64
 19  feature_20  28240 non-null  float64
 20  feature_21  23258 non-null  float64
 21  feature_22  26095 non-null  float64
 22  feature_23  28240 non-null  float64
 23  feature_24  38985 non-null  int64
 24  feature_25  38985 non-null  int64
 25  feature_26  38985 non-null  int64
 26  feature_27  38985 non-null  int64
 27  feature_28  38985 non-null  int64
 28  feature_29  38985 non-null  int64
 29  feature_30  38985 non-null  int64
 30  feature_31  38985 non-null  int64
 31  feature_32  38985 non-null  int64
 32  feature_33  38985 non-null  int64
 33  feature_34  38985 non-null  int64
 34  feature_35  38985 non-null  int64
 35  feature_36  18880 non-null  float64
 36  feature_37  16739 non-null  float64
 37  feature_38  15413 non-null  float64
 38  feature_39  24684 non-null  float64
```

```
 39  feature_40  24684 non-null  float64
 40  feature_41  24684 non-null  float64
 41  feature_42  24684 non-null  float64
 42  feature_43  24684 non-null  float64
 43  feature_44  24684 non-null  float64
 44  feature_45  24684 non-null  float64
 45  feature_46  24684 non-null  float64
 46  feature_47  24684 non-null  float64
 47  feature_48  24684 non-null  float64
 48  feature_49  24684 non-null  float64
 49  feature_50  8498 non-null   float64
 50  feature_51  24684 non-null  float64
 51  feature_52  24684 non-null  float64
```

```
df.shape
```

```
(38985, 87)
```

```
df.columns
```

```
Index(['feature_1', 'feature_2', 'feature_3', 'feature_4', 'feature_5',
       'feature_6', 'feature_7', 'feature_8', 'feature_9', 'feature_10',
       'feature_11', 'feature_12', 'feature_13', 'feature_14', 'feature_15',
       'feature_16', 'feature_17', 'feature_18', 'feature_19', 'feature_20',
       'feature_21', 'feature_22', 'feature_23', 'feature_24', 'feature_25',
       'feature_26', 'feature_27', 'feature_28', 'feature_29', 'feature_30',
       'feature_31', 'feature_32', 'feature_33', 'feature_34', 'feature_35',
       'feature_36', 'feature_37', 'feature_38', 'feature_39', 'feature_40',
       'feature_41', 'feature_42', 'feature_43', 'feature_44', 'feature_45',
       'feature_46', 'feature_47', 'feature_48', 'feature_49', 'feature_50',
       'feature_51', 'feature_52', 'feature_53', 'feature_54', 'feature_55',
       'feature_56', 'feature_57', 'feature_58', 'feature_59', 'feature_60',
       'feature_61', 'feature_62', 'feature_63', 'feature_64', 'feature_65',
       'feature_66', 'feature_67', 'feature_68', 'feature_69', 'feature_70',
       'feature_71', 'feature_72', 'feature_73', 'feature_74', 'feature_75',
       'feature_76', 'feature_77', 'feature_78', 'feature_79', 'feature_80',
       'feature_81', 'feature_82', 'feature_83', 'feature_84', 'feature_85',
       'ID', 'target'],
      dtype='object')
```

```
df.describe(include='all')
```

	feature_1	feature_2	feature_3	feature_4	feature_5	feature_6	feature_7	feature_8	feature_9
count	3.898500e+04	38985.000000	38985.000000	38985.000000	36771.000000	36771.000000	36771.000000	38985.000000	38985.000000
unique	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
top	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
freq	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
mean	3.484706e+04	45.646197	0.482801	0.524484	600.987651	238.338737	206.265718	6.850045	10.143517
std	3.406895e+04	13.399512	0.499711	0.514086	572.164116	335.988411	392.821775	7.269573	18.639226
min	3.000000e+03	18.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.530924e+04	35.000000	0.000000	0.000000	261.210000	28.428600	0.000000	2.000000	3.000000
50%	2.500000e+04	45.000000	0.000000	1.000000	392.000000	108.571400	0.000000	5.000000	6.000000
75%	4.311684e+04	56.000000	1.000000	1.000000	755.500000	269.142900	226.142900	9.000000	12.000000
max	2.081718e+06	76.000000	1.000000	3.000000	4458.310000	2239.428600	2378.714300	186.000000	953.000000

11 rows × 87 columns

```
duplicates = df.duplicated().sum()
print(duplicates)
```

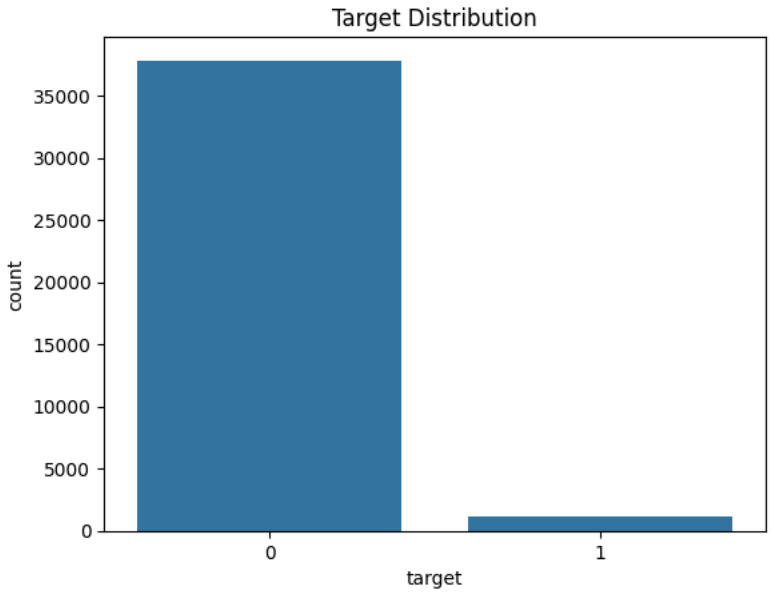
```
0
```

```
# target distribution shows data is severely imbalanced
df['target'].value_counts()
df['target'].value_counts(normalize=True)
```

target	proportion
0	0.971066
1	0.028934

dtype: float64

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.countplot(x='target', data=df)
plt.title("Target Distribution")
plt.show()
```




```
#missing value percentage
missing_pct = (df.isna().sum()/ len(df) * 100).sort_values(ascending=False)
#print(missing_val)
missing_pct.head(50)
#len(missing_val > 0)
```

	0
feature_50	78.201873
feature_38	60.464281
feature_37	57.062973
feature_36	51.571117
feature_56	47.567013
feature_66	46.289599

2.Split the data

```
# separating independent and dependent variables, ID removed because it is not predictive
X = df.drop(columns=['target', 'ID'])
y = df['target']
```

```
# creating test set to avoid overfitting and data leakage
#15% for test data
from sklearn.model_selection import train_test_split

X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y,
    test_size=0.15,
    stratify=y,
    random_segment>
)
```

```
# creating validation test
# 15% for validation and 70% for train data
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val,
    test_size=0.176,
    stratify=y_train_val,
    random_state=42
)
```

feature_52	36.683340

```
#checking if stratification worked
print("Train:")
print(y_train.value_counts(normalize=True))

print("\nValidation:")
print(y_val.value_counts(normalize=True))

print("\nTest:")
print(y_test.value_counts(normalize=True))
```

feature_20	27.561883
feature_23	27.561883
feature_43	5.679107
feature_7	5.679107
feature_6	5.679107
feature_5	5.679107
feature_14	5.679107
feature_10	5.679107
feature_1	0.000000
feature_4	0.000000
feature_11	0.000000
feature_9	0.000000

```
Train:
target
0    0.971067
1    0.028933
Name: proportion, dtype: float64

Validation:
target
0    0.971027
1    0.028973
Name: proportion, dtype: float64

Test:
target
0    0.971101
1    0.028899
Name: proportion, dtype: float64
```

3. Feature Selection

```
# learning what to drop from training data only, dropping features with > 35% missingness
missing_pct_train = X_train.isna().sum() / len(X_train) * 100
cols_to_drop = missing_pct_train[missing_pct_train > 35].index.tolist()
cols_to_drop
```

feature_2	0.000000
feature_52	0.000000

```
['feature_15',
 'feature_18',
```

```
 'feature_21',
 'feature_33',
 'feature_36',
 'feature_37',
 'feature_38',
 'feature_39',
 'feature_40',
 'feature_41',
 'feature_42',
 'feature_43',
 'feature_44',
 'feature_45',
 'feature_46',
 'feature_47',
 'feature_48',
 'feature_49',
 'feature_50',
 'feature_51',
 'feature_52',
 'feature_53',
 'feature_54',
 'feature_55',
 'feature_56',
 'feature_66']
```

```
feature_33    0.000000
feature_27    0.000000
feature_26    0.000000
feature_25    0.000000
dtype: float64
```

```
train_missing = (X_train.isna().sum() / len(X_train) * 100).sort_values(ascending=False)
train_missing.head(13)
```

	0
feature_50	78.186346
feature_38	60.320832
feature_37	56.962350
feature_36	51.516261
feature_56	47.699971
feature_66	46.747729
feature_15	40.265163
feature_21	40.265163
feature_18	40.265163
feature_49	36.690595
feature_51	36.690595
feature_44	36.690595
feature_41	36.690595

dtype: float64

```
X_train = X_train.drop(columns=cols_to_drop)
X_val   = X_val.drop(columns=cols_to_drop)
X_test  = X_test.drop(columns=cols_to_drop)

print("Remaining features:", X_train.shape[1])
```

```
Remaining features: 60
```

```
#imputing the rest of the features on training data that have < 35% missingness to prevent errors in model training
from sklearn.impute import SimpleImputer
import pandas as pd

feature_names = X_train.columns

imputer = SimpleImputer(strategy="median")

X_train_imp = imputer.fit_transform(X_train)

# Transform validation and test using the same imputer
X_val_imp   = imputer.transform(X_val)
X_test_imp  = imputer.transform(X_test)

X_train = pd.DataFrame(X_train_imp, columns=feature_names, index=X_train.index)
X_val   = pd.DataFrame(X_val_imp,   columns=feature_names, index=X_val.index)
```

```
X_test  = pd.DataFrame(X_test_imp,  columns=feature_names, index=X_test.index)

# checking for na values
print("Any NaNs left in train?", X_train.isna().sum().sum())
print("Any NaNs left in val?", X_val.isna().sum().sum())
print("Any NaNs left in test?", X_test.isna().sum().sum())
```

```
Any NaNs left in train? 0
Any NaNs left in val? 0
Any NaNs left in test? 0
```

```
# variance threshold method for feature selection to remove features with near 0 variance
from sklearn.feature_selection import VarianceThreshold
selector = VarianceThreshold(threshold = 0.01)
X_var = selector.fit_transform(X_train)

selected_features = X_train.columns[selector.get_support()]
print('Remaining Features:', (selected_features))
print(len(selected_features), 'features left')
```

```
Remaining Features: Index(['feature_1', 'feature_2', 'feature_3', 'feature_4', 'feature_5',
       'feature_6', 'feature_7', 'feature_8', 'feature_9', 'feature_10',
       'feature_11', 'feature_12', 'feature_13', 'feature_14', 'feature_16',
       'feature_17', 'feature_19', 'feature_20', 'feature_22', 'feature_23',
       'feature_24', 'feature_25', 'feature_26', 'feature_27', 'feature_28',
       'feature_29', 'feature_30', 'feature_31', 'feature_32', 'feature_33',
       'feature_34', 'feature_35', 'feature_57', 'feature_60', 'feature_61',
       'feature_62', 'feature_64', 'feature_65', 'feature_71', 'feature_72',
       'feature_74', 'feature_75', 'feature_76', 'feature_77', 'feature_78',
       'feature_79', 'feature_80', 'feature_81', 'feature_83', 'feature_84',
       'feature_85'],
      dtype='object')
51 features left
```

```
X_train = X_train[selected_features]
X_val   = X_val[selected_features]
X_test  = X_test[selected_features]
```

```
#ANOVA method to identify features whose average values differ between classes
from sklearn.feature_selection import SelectKBest, f_classif
import pandas as pd

selector = SelectKBest(score_func=f_classif, k='all')
selector.fit(X_train, y_train)

anova_scores = pd.Series(selector.scores_, index=X_train.columns)
anova_scores = anova_scores.sort_values(ascending=False)

anova_scores.head(15)
```

	0
feature_71	55.793584
feature_2	37.475271
feature_72	33.587922
feature_10	31.095094
feature_8	27.714920
feature_7	25.483253
feature_81	24.313791
feature_84	22.058987
feature_74	20.866118
feature_28	19.941446
feature_6	19.638081
feature_14	18.968656
feature_34	18.874377
feature_17	16.540877
feature_85	16.337376

dtype: float64

```
#random forest method to identify features with predictive accuracy
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

rf_importance = pd.Series(rf.feature_importances_, index=X_train.columns)
rf_importance = rf_importance.sort_values(ascending=False)

rf_importance.head(15)
```

	0
feature_1	0.051092
feature_11	0.043456
feature_10	0.043088
feature_2	0.042991
feature_57	0.042326
feature_6	0.041795
feature_14	0.040278
feature_13	0.039477
feature_5	0.038669
feature_29	0.032587
feature_17	0.032112
feature_20	0.031010
feature_23	0.029847
feature_35	0.028178
feature_16	0.027116

dtype: float64

```
#identifying overlapping top (most relevant) features between anova and random forest method of selection
anova_top = set(anova_scores.head(20).index)
rf_top = set(rf_importance.head(20).index)

important_features = list(anova_top & rf_top)
```

```
important_features
```

```
['feature_10',
 'feature_8',
 'feature_14',
 'feature_7',
 'feature_16',
 'feature_6',
 'feature_57',
 'feature_17',
 'feature_2']
```

4. Train multiple classification models

```
#scaling data
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled   = scaler.transform(X_val)
X_test_scaled  = scaler.transform(X_test)
```

```
# validating logistic regression performance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score

log_model = LogisticRegression(max_iter=1000, random_state=42)
log_model.fit(X_train_scaled, y_train)

log_pred = log_model.predict(X_val_scaled)
log_prob = log_model.predict_proba(X_val_scaled)[:,1]

print("Logistic Accuracy:", accuracy_score(y_val, log_pred))
print("Logistic ROC-AUC:", roc_auc_score(y_val, log_prob))
```

```
Logistic Accuracy: 0.9710269158237613
Logistic ROC-AUC: 0.6440385451141644
```

```
#random forest
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_val)
rf_prob = rf_model.predict_proba(X_val)[:,1]

print("RF Accuracy:", accuracy_score(y_val, rf_pred))
print("RF ROC-AUC:", roc_auc_score(y_val, rf_prob))
```

```
RF Accuracy: 0.9710269158237613
RF ROC-AUC: 0.6001759268545448
```

```
# xg boost
!pip install xgboost
from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    n_estimators=500,
    max_depth=3,
    learning_rate=0.03,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    eval_metric='logloss'
)

xgb_model.fit(X_train, y_train)
```

```
xgb_pred = xgb_model.predict(X_val)
xgb_prob = xgb_model.predict_proba(X_val)[:,1]

print("XGB Accuracy:", accuracy_score(y_val, xgb_pred))
print("XGB ROC-AUC:", roc_auc_score(y_val, xgb_prob))
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.3)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
XGB Accuracy: 0.9710269158237613
XGB ROC-AUC: 0.6502523986226725
```

```
# support vector machine (SVM)
from sklearn.svm import SVC

svm_model = SVC(
    kernel='rbf',
    probability=True,
    random_state=42
)

svm_model.fit(X_train_scaled, y_train)

svm_pred = svm_model.predict(X_val_scaled)
svm_prob = svm_model.predict_proba(X_val_scaled)[:,1]

print("SVM Accuracy:", accuracy_score(y_val, svm_pred))
print("SVM ROC-AUC:", roc_auc_score(y_val, svm_prob))
```

```
SVM Accuracy: 0.9710269158237613
SVM ROC-AUC: 0.5537287299836191
```

```
# comparing classification model performances
results = {
    "Logistic": roc_auc_score(y_val, log_prob),
    "Random Forest": roc_auc_score(y_val, rf_prob),
    "XGBoost": roc_auc_score(y_val, xgb_prob),
    "SVM": roc_auc_score(y_val, svm_prob)
}

results
```

```
{'Logistic': np.float64(0.6440385451141644),
 'Random Forest': np.float64(0.6001759268545448),
 'XGBoost': np.float64(0.6502523986226725),
 'SVM': np.float64(0.5537287299836191)}
```

```
# evaluating test set using xg boost, which had the best performance

from sklearn.metrics import accuracy_score, roc_auc_score, classification_report

test_pred = xgb_model.predict(X_test)
test_prob = xgb_model.predict_proba(X_test)[:,1]

print("Test Accuracy:", accuracy_score(y_test, test_pred))
print("Test ROC-AUC:", roc_auc_score(y_test, test_prob))
print(classification_report(y_test, test_pred))
```

```
Test Accuracy: 0.9711012311901505
Test ROC-AUC: 0.6810474800234643
              precision    recall  f1-score   support

           0       0.97      1.00      0.99      5679
           1       0.00      0.00      0.00       169

    accuracy                           0.97      5848
   macro avg       0.49      0.50      0.49      5848
weighted avg       0.94      0.97      0.96      5848

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

5. Parameter optimization

```
from sklearn.metrics import roc_auc_score

def eval_auc(model, X_val_use, y_val):
    prob = model.predict_proba(X_val_use)[:, 1]
    return roc_auc_score(y_val, prob)
```

```
# logistic regression tuned
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Baseline
log_base = LogisticRegression(max_iter=2000, random_state=42)
log_base.fit(X_train_scaled, y_train)
log_base_auc = eval_auc(log_base, X_val_scaled, y_val)

# Tune
log_params = {
    "C": [0.01, 0.1, 1, 10],
    "penalty": ["l2"],
    "solver": ["lbfgs"]
}
log_grid = GridSearchCV(
    LogisticRegression(max_iter=2000, random_state=42),
    log_params,
    scoring="roc_auc",
    cv=3,
    n_jobs=-1
)
log_grid.fit(X_train_scaled, y_train)

log_best = log_grid.best_estimator_
log_tuned_auc = eval_auc(log_best, X_val_scaled, y_val)

log_grid.best_params_, log_base_auc, log_tuned_auc
```

```
({'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'},
 np.float64(0.6440385451141644),
 np.float64(0.6539809196670344))
```

```
# random forest tuned
from sklearn.ensemble import RandomForestClassifier

# Baseline
rf_base = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
rf_base.fit(X_train, y_train)
rf_base_auc = eval_auc(rf_base, X_val, y_val)

# Tune
rf_params = {
    "n_estimators": [200, 400],
    "max_depth": [None, 8, 15],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}
rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42, n_jobs=-1),
    rf_params,
    scoring="roc_auc",
    cv=3,
    n_jobs=-1
)
rf_grid.fit(X_train, y_train)

rf_best = rf_grid.best_estimator_
rf_tuned_auc = eval_auc(rf_best, X_val, y_val)

rf_grid.best_params_, rf_base_auc, rf_tuned_auc
```

```
({'max_depth': 8,
  'min_samples_leaf': 1,
  'min_samples_split': 5,
  'n_estimators': 400},
```

```
 np.float64(0.6001759268545448),
 np.float64(0.6443362835556448))
```

```
# SVM tuned
from sklearn.svm import SVC

# Baseline
svm_base = SVC(kernel="rbf", probability=True, random_state=42)
svm_base.fit(X_train_scaled, y_train)
svm_base_auc = eval_auc(svm_base, X_val_scaled, y_val)

# Tune
svm_params = {
    "C": [0.5, 1, 5, 10],
    "gamma": ["scale", 0.1, 0.01],
    "kernel": ["rbf"]
}
svm_grid = GridSearchCV(
    SVC(probability=True, random_state=42),
    svm_params,
    scoring="roc_auc",
    cv=3,
    n_jobs=-1
)
svm_grid.fit(X_train_scaled, y_train)

svm_best = svm_grid.best_estimator_
svm_tuned_auc = eval_auc(svm_best, X_val_scaled, y_val)

svm_grid.best_params_, svm_base_auc, svm_tuned_auc
```

```
/usr/local/lib/python3.12/dist-packages/joblib/externals/loky/process_executor.py:782: UserWarning: A worker stopped
  warnings.warn(
({'C': 5, 'gamma': 'scale', 'kernel': 'rbf'},
 np.float64(0.5537287299836191),
 np.float64(0.5547817838398021))
```

```
# XG Boost tuning
from xgboost import XGBClassifier

# Baseline (use whatever you used before tuning; here is a reasonable baseline)
xgb_base = XGBClassifier(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    eval_metric="logloss"
)
xgb_base.fit(X_train, y_train)
xgb_base_auc = eval_auc(xgb_base, X_val, y_val)

# Tuned (your best params from GridSearch)
xgb_best = XGBClassifier(
    colsample_bytree=0.8,
    learning_rate=0.01,
    max_depth=3,
    n_estimators=600,
    subsample=0.7,
    random_state=42,
    eval_metric="logloss"
)
xgb_best.fit(X_train, y_train)
xgb_tuned_auc = eval_auc(xgb_best, X_val, y_val)

xgb_base_auc, xgb_tuned_auc
```

```
(np.float64(0.6275072710861499), np.float64(0.6619247902249857))
```

```
#impact on performance
import pandas as pd

results = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest", "SVM (RBF)", "XGBoost"],
    "Baseline ROC-AUC (val)": [log_base_auc, rf_base_auc, svm_base_auc, xgb_base_auc],
```

```
    "Tuned ROC-AUC (val)": [log_tuned_auc, rf_tuned_auc, svm_tuned_auc, xgb_tuned_auc],
})

results["Δ ROC-AUC"] = results["Tuned ROC-AUC (val)"] - results["Baseline ROC-AUC (val)"]
results.sort_values("Tuned ROC-AUC (val)", ascending=False)
```

	Model	Baseline ROC-AUC (val)	Tuned ROC-AUC (val)	Δ ROC-AUC
3	XGBoost	0.627507	0.661925	0.034418
0	Logistic Regression	0.644039	0.653981	0.009942
1	Random Forest	0.600176	0.644336	0.044160
2	SVM (RBF)	0.553729	0.554782	0.001053

6. Addressing class balance

```
# adding class weights
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
scale_pos_weight
```

```
np.float64(33.562025316455696)
```

```
xgb_weighted = XGBClassifier(
    colsample_bytree=0.8,
    learning_rate=0.01,
    max_depth=3,
    n_estimators=600,
    subsample=0.7,
    scale_pos_weight=scale_pos_weight,
    random_state=42,
    eval_metric='logloss'
)

xgb_weighted.fit(X_train, y_train)
```

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.01, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=3, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=600, n_jobs=None,
              num_parallel_tree=None, ...)
```

```
from sklearn.metrics import classification_report, roc_auc_score

pred = xgb_weighted.predict(X_test)
prob = xgb_weighted.predict_proba(X_test)[:,1]

print("ROC-AUC:", roc_auc_score(y_test, prob))
print(classification_report(y_test, pred))
```

```
ROC-AUC: 0.6770589454973217
              precision    recall  f1-score   support

           0       0.98      0.73      0.84      5679
           1       0.06      0.56      0.11       169

    accuracy                           0.72      5848
   macro avg       0.52      0.65      0.47      5848
weighted avg       0.96      0.72      0.82      5848
```

```
# SMOTE oversampling
!pip -q install imbalanced-learn
from imblearn.over_sampling import SMOTE
from sklearn.metrics import classification_report, roc_auc_score
from xgboost import XGBClassifier
```

```
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print("Before SMOTE:\n", y_train.value_counts())
print("\nAfter SMOTE:\n", y_train_sm.value_counts())
```

```
Before SMOTE:
 target
0    26514
1      790
Name: count, dtype: int64

After SMOTE:
 target
0    26514
1    26514
Name: count, dtype: int64
```

```
xgb_smote = XGBClassifier(
    colsample_bytree=0.8,
    learning_rate=0.01,
    max_depth=3,
    n_estimators=600,
    subsample=0.7,
    random_state=42,
    eval_metric="logloss"
)

xgb_smote.fit(X_train_sm, y_train_sm)
```

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.01, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=3, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=600, n_jobs=None,
              num_parallel_tree=None, ...)
```

```
pred_sm = xgb_smote.predict(X_test)
prob_sm = xgb_smote.predict_proba(X_test)[:, 1]

print("ROC-AUC:", roc_auc_score(y_test, prob_sm))
print(classification_report(y_test, pred_sm))

# SMOTE did not work as effectively as class weights
```

```
ROC-AUC: 0.5902468452754933
              precision    recall  f1-score   support

           0       0.97      1.00      0.99      5679
           1       0.00      0.00      0.00       169

    accuracy                           0.97      5848
   macro avg       0.49      0.50      0.49      5848
weighted avg       0.94      0.97      0.96      5848
```

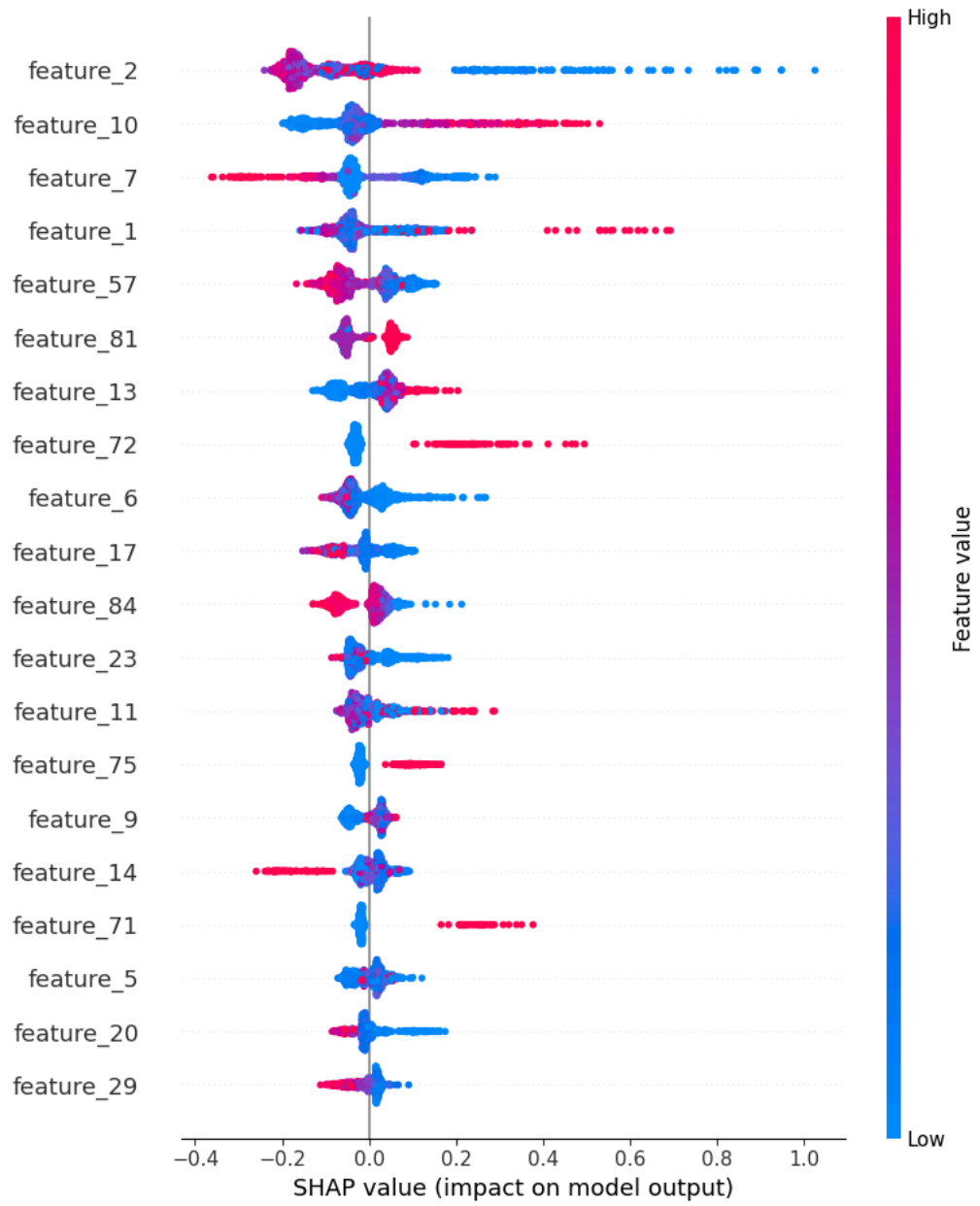
7. Global explainability

```
# SHAP to explain model behavior
!pip -q install shap
import shap
```

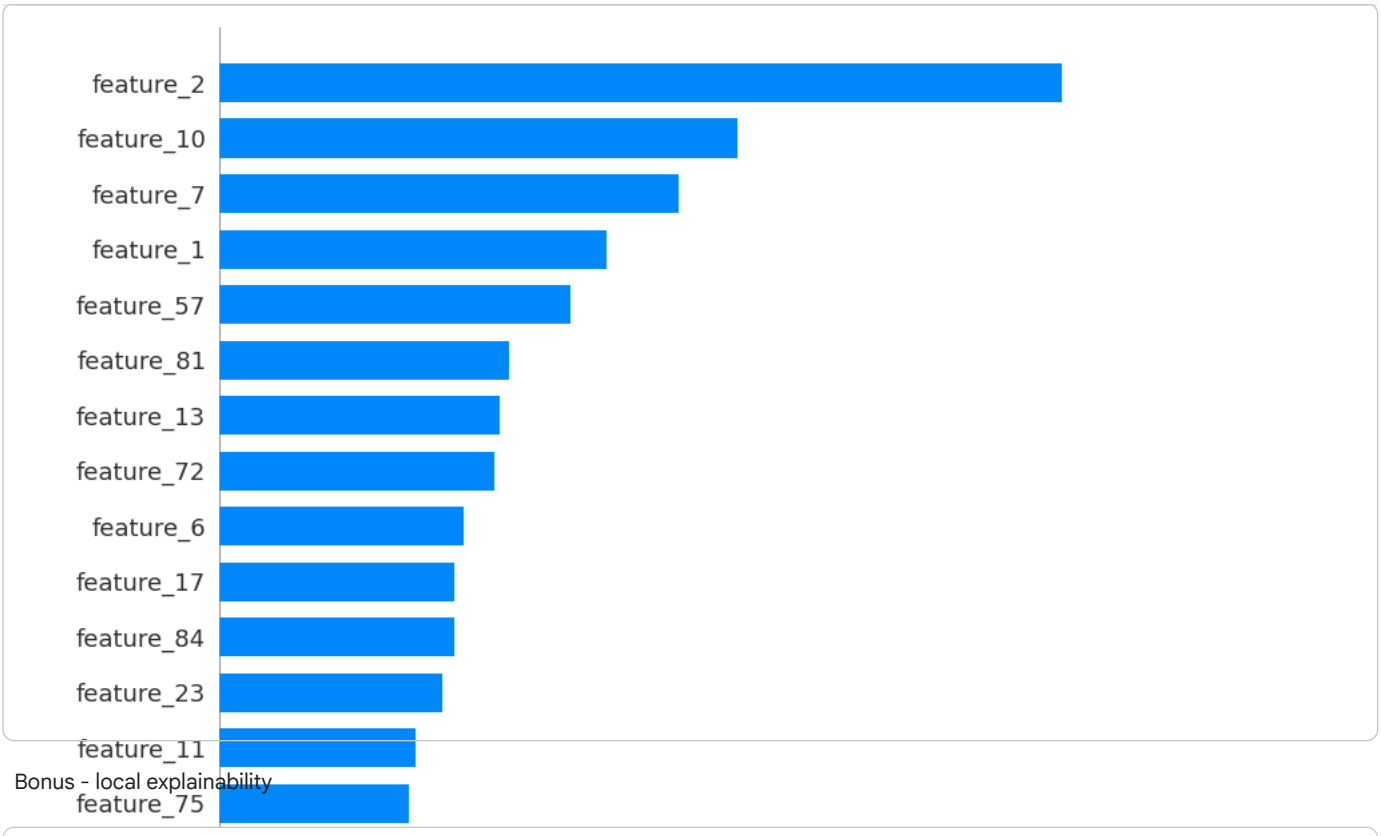
```
# using xg boost to create SHAP explainer
explainer = shap.TreeExplainer(xgb_best)
# using sample for speed
sample = X_test.sample(1000, random_state=42)

shap_values = explainer.shap_values(sample)
```

```
shap.summary_plot(shap_values, sample)
```




```
# bar plot visualizing feature ranking with shap values
shap.summary_plot(shap_values, sample, plot_type="bar")
```



Bonus - local explainability

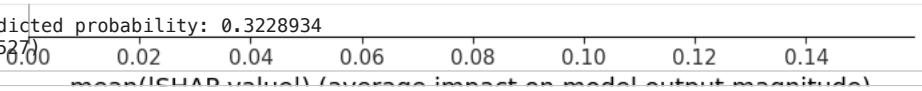
```
 #selecting one observation from the test set and highest risk case
import numpy as np

# predicted probabilities for class 1 (default)
test_prob = xgb_best.predict_proba(X_test)[:, 1]

# picking the index with the highest predicted risk
high_risk_pos = np.argmax(test_prob)
high_risk_index = X_test.index[high_risk_pos]

print("Highest predicted probability:", test_prob[high_risk_pos])
high_risk_index
```

```
Highest predicted probability: 0.3228934
np.int64(10527)
```




```
x_one = X_test.loc[[high_risk_index]]
shap_values_one = explainer.shap_values(x_one)
```

```
import shap

shap.plots.waterfall(
    shap.Explanation(
        values=shap_values_one[0],
        base_values=explainer.expected_value
```